Evaluating Low-Rank / Tensor Decomposition of the LLaMA-3.2 KV Cache

A Beginner's Project Handout — Single Device

Round-trip latency • end-to-end latency • memory • energy • accuracy

What you will be able to say at the end: "On LLaMA-3.2-1B/3B, on one GPU, low-rank (rank-r) decomposition of the KV cache compresses keys/values by X%; it costs Y perplexity / Z% task accuracy; and after reconstruction overhead it is faster / slower by W% per token — here is the compression-vs-accuracy-vs-latency frontier, and here is which of keys, values, and layers are actually compressible."

Written against your environment: **transformers 4.57.6, torch 2.8.0**. All architecture numbers, API calls, and evaluation protocols are verified (sources cited). Code is runnable; copy it stage by stage.

Project handout • July 22, 2026

Contents

How to use this handout **2**

Part I — Concepts you need first **2**

1 The KV cache, exactly **2**
1.1 Grouped-Query Attention (GQA): get the shape right 2
1.2 Per-token KV memory — the formula you will quote constantly 2

2 What "tensor / low-rank decomposition" means here **3**
2.1 Matrix low rank (start here) 3
2.2 Tensor decompositions, and three granularities 3

3 The one conceptual trap: measure the right error **4**

Part II — The experiment **4**

4 Question, scope, variables **4**

Part III — Metrics and how to measure them **5**

5 Definitions and measurement recipes **5**

Part IV — The staged plan **6**

6 Milestones (do them in order) **6**

Part V — Implementation **7**

7 Environment **7**

8 Load the model **8**

9 Stage 1 — extract K/V and plot the spectrum **8**

10 Stages 2–3 — a low-rank cache **8**

11 Perplexity (baseline quality, Stage 0) **9**

12 Stage 5 — per-layer Tucker (granularity 3, the tensor step-up) **10**

Part VI — Interpreting your results **10**

13 What to expect (and the honest lessons) **10**

Part VII — Related work **11**

14 The landscape (grouped, with code where public) **11**

Part VIII — Deliverables & checklist **12**

15 What to produce **12**

16 Reproducibility checklist **12**

How to use this handout

Read **Part I** (concepts) once. Then work **Part IV** stage by stage — each stage has a goal, the code you need (Part V), and what to expect (Part VI). **Part III** is your reference for *how to measure each metric correctly*; skim it before Stage 3 and return to it whenever you report a number. **Part VII** is the literature you are building on. If you only do one thing first, do **Stage 1** (the SVD spectrum study) — it is cheap, needs no attention surgery, and tells you whether the whole premise even holds on LLaMA-3.2.

Part I — Concepts you need first

1 The KV cache, exactly

During generation a transformer produces one token at a time; at each step every layer must attend to *all previous tokens*. To avoid recomputing them, each layer stores the **keys** (K) and **values** (V) of all past tokens — the **KV cache**. It grows by one entry per layer per generated token, so it grows *linearly with context length* and, at long context, becomes larger than the model weights (Table 2).

1.1 Grouped-Query Attention (GQA): get the shape right

LLaMA-3.2 uses **Grouped-Query Attention**: there are many *query* heads but only a few *key/value* heads, which the query heads share. **The cache stores the KV heads, not the query heads** — this is the #1 thing beginners get wrong. Per layer, for K and for V:

$$\text{shape} = (\text{batch}, \underbrace{\text{num_key_value_heads}}_{=8 \text{ for both 1B and 3B}}, \text{seq_len}, \text{head_dim}).$$

	Llama-3.2-1B	**Llama-3.2-3B**	(verified from config.json)
decoder layers	16	28	
hidden size	2048	3072	
query heads	32	24	
KV heads (GQA)	**8**	**8**	cache uses this
head_dim	64	128	
FFN size	8192	8192	
vocab	128256	128256	tied embeddings
max context	131072	131072	
default dtype	bfloat16	bfloat16	2 bytes/element

Table 1: Verified LLaMA-3.2 architecture (base and `-Instruct` are identical except the EOS token id).

1.2 Per-token KV memory — the formula you will quote constantly

$$\text{KV bytes per token} = 2\ (\text{K and V}) \times \text{layers} \times \text{KV heads} \times \text{head_dim} \times \text{bytes/element}.$$

For bf16 (2 bytes): **1B** $= 2 \times 16 \times 8 \times 64 \times 2 = 32{,}768$ bytes $= 32$ KiB/token; **3B** $= 2 \times 28 \times 8 \times 128 \times 2 = 114{,}688$ bytes $= 112$ KiB/token.

context (tokens)	1K	4K	8K	32K	128K
1B KV size (32 KiB/tok)	32 MiB	128 MiB	256 MiB	1.0 GiB	4.0 GiB
3B KV size (112 KiB/tok)	112 MiB	448 MiB	896 MiB	3.5 GiB	14.0 GiB
weights (bf16)	1B ≈ 2.3 GiB		3B ≈ 6.0 GiB		

Table 2: KV cache size vs context (batch 1). At 128K the KV cache exceeds the weights; under batched serving it dominates even sooner. *This* is why the KV cache is the thing to compress.

2 What "tensor / low-rank decomposition" means here

2.1 Matrix low rank (start here)

Take one layer, one KV head. Its key cache is a matrix $K \in \mathbb{R}^{S \times d}$ (S tokens, $d = \text{head_dim}$). A **rank-r approximation** writes $K \approx AB^\top$ with $A \in \mathbb{R}^{S \times r}$, $B \in \mathbb{R}^{d \times r}$ and $r < d$. The best such approximation (in Frobenius norm) comes from the **truncated SVD**: $K = U\Sigma V^\top$, keep the top r singular values. Storage drops from Sd numbers to $r(S + d)$ — a win when $r \ll d$. **Compression ratio** $= Sd/(r(S + d))$.

Key idea

"Low-rank" works only if the singular values of K decay fast — i.e. a few directions carry most of the energy. **Whether that is true for LLaMA's KV is an empirical question** and is exactly what Stage 1 measures. Spoiler from the literature: keys tend to be low-rank, *values much less so* [12] — verify it yourself.

2.2 Tensor decompositions, and three granularities

The full per-layer cache is a 3-D **tensor** (heads × tokens × head_dim). **Tucker**, **CP/PARAFAC**, and **Tensor-Train (TT)** decompositions [20, 21] compress *several axes jointly* rather than one matrix at a time; the Python library `tensorly` implements all three. You will meet three natural granularities:

1. **per-(layer, head)** — SVD of each $[S \times d]$ matrix (Stages 1–4). Finest and most interpretable; where the low-rank structure demonstrably is.
2. **per-layer, heads flattened** — SVD of one $[S \times (H \cdot d)]$ matrix per layer; captures cross-head redundancy with a single knob.
3. **per-layer 3-D tensor** — Tucker/TT of $[H \times S \times d]$ (Stage 5); compresses the head mode jointly — the genuinely "tensor" contribution (cf. JoLT [15], DecoQuant [6]).

Start with granularity (1), then step up to (3) once it works.

Key idea

Do all of this *per layer*. The layer axis is for **rank allocation, not decomposition**: stack per-layer results only to *budget* ranks across layers (protect the least-compressible ones), never as a joint decomposition over the layer axis. Different layers are produced by different projection weights $W_K^{(\ell)}$ and share no common basis, so a cross-layer factorization saves nothing at attention time (which is per-layer anyway) and only confounds the depth-dependence you are trying to measure.

3 The one conceptual trap: measure the right error

The obvious thing to minimize is the **reconstruction error** $\|K - \hat{K}\|_F$. **This is the wrong target.** The cache exists only to produce the attention output $o = \text{softmax}(qK^\top/\sqrt{d})\,V$; a large $\|K - \hat{K}\|$ in a direction the query never looks at is harmless, and a tiny error on a high-attention token can flip the output. The peer-reviewed statement of this is KDEformer [16]: prior methods minimized $\|A - \tilde{A}\|_F$ "even though A is not the ultimate output of attention and the output depends on V."

Key idea

Report **attention-output error** and **end-task accuracy**, not just $\|K - \hat{K}\|_F$. Use reconstruction error / the singular-value spectrum as a *diagnostic* (Stage 1), but judge quality by the output (Stages 2, 4). Recent theory ties the output error to the *missed attention mass* [18, 17], not to raw KV error.

Common pitfall

You may read that "softmax *amplifies* error when attention is concentrated." Taken literally that is *backwards*: the softmax Jacobian $\text{diag}(p) - pp^\top$ shrinks as attention peaks, so softmax is *least* sensitive to logit noise when concentrated. The real reasons output error $\neq$ reconstruction error are (i) value error is weighted by attention probability ($o = \sum_i p_i v_i$, so $\partial o/\partial v_i = p_i$ — error on high-attention tokens matters more); (ii) dropping/mis-estimating attention mass then renormalizing shifts the output [18]; and (iii) errors compound over autoregressive steps [5]. State it this way and you will be correct.

Part II — The experiment

4 Question, scope, variables

Research question. On a single device, how does rank-r (and later Tucker/TT) decomposition of the LLaMA-3.2 KV cache trade *compression* against *accuracy* and *latency/memory/energy* — and *which* parts of the cache (keys vs values, shallow vs deep layers) are actually compressible?

- **Model:** start with **Llama-3.2-1B** (16 layers — fast to iterate, fits any modern GPU), then repeat on **Llama-3.2-3B** (28 layers — matches the FSU poster [1]). Use the `-Instruct` variants for task accuracy.
- **Baseline:** the unmodified model with a full bf16 KV cache.
- **Method under test:** truncated-SVD rank-r reconstruction of K and V, per (layer, head); sweep r. Extension: Tucker/TT over the (head × token × feature) tensor.
- **Independent variables:** rank r (hence compression ratio); context length; keys-only vs values-only vs both; per-layer rank.
- **The three sub-questions:** **Q1** does the low-rank premise hold (Stage 1)? **Q2** what does it cost accuracy (Stages 2, 4)? **Q3** is it actually faster/smaller after overhead (Stage 3)?

Tip

Two experiments answer two different questions and must not be conflated. **Accuracy** experiments *reconstruct* $\hat{K}, \hat{V}$ (dense) and run normal attention — this measures quality but saves no memory and *adds* SVD time. **Memory/latency** experiments must *store the factors* A, B (memory saved) and reconstruct on the fly (compute added), or compute attention directly from factors (the research frontier). Keep the two separate; §13 explains why the naive version can be *slower*.

Part III — Metrics and how to measure them correctly

5 Definitions and measurement recipes

Metric	Definition	Tool / pitfall
TTFT (time-to-first-token)	latency of the prefill pass (prompt → first token)	generate 1 token; `cuda.synchronize`
TPOT / per-step ("round time")	steady-state cost of one decode step = (E2E − TTFT)/(N−1)	CUDA events; report median & P95
End-to-end latency	full generation time for N tokens	warmup first; fix seed
Throughput	tokens / second	= N/E2E
Peak memory	max GPU bytes during run	`max_memory_allocated`; reset first
KV bytes	cache size	analytic formula *and* measured delta
Energy / power	Joules or Watts per token	NVML counter (desktop); `tegrastats` (Jetson)
Compression ratio	dense bytes / compressed bytes	state keys-only vs both
Reconstruction err. (diagnostic)	$\|K - \hat{K}\|_F / \|K\|_F$	*diagnostic only*
Attention-output err.	$\|o - \hat{o}\|$, or KL of next-token dist.	the error that matters
Perplexity	exp(mean NLL) on WikiText-2	strided window [32]
Task accuracy	LongBench / RULER / GSM8K	GSM8K guards reasoning

Table 3: The metric kit. Report accuracy vs compression as a Pareto curve, with latency/memory/energy alongside.

Common pitfall

CUDA is asynchronous. A bare `time.perf_counter()` around `model(...)` times only the *kernel launch*, not the work — you will report numbers that are wildly too small. Always `torch.cuda.synchronize()` around wall-clock timing, or use `torch.cuda.Event` (below). Also: do **warmup** iterations (first calls pay CUDA-context / cuBLAS / autotune costs), and report **median and P95**, not mean — decode latency is right-skewed.

The measurement helpers (use these everywhere):

```python
import torch, numpy as np, time

def time_ms(fn, warmup=10, iters=50):
    """Median/P95 latency of fn() in milliseconds, done correctly."""
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()
    ts = []
    for _ in range(iters):
        s = torch.cuda.Event(enable_timing=True); e = torch.cuda.Event(enable_timing=True)
        s.record(); fn(); e.record()
        torch.cuda.synchronize()          # REQUIRED before reading elapsed_time
        ts.append(s.elapsed_time(e))
    t = np.array(ts)
    return {"median": float(np.median(t)), "p95": float(np.percentile(t, 95))}

def kv_bytes_measured(model, inputs):
    """Isolate the KV-cache cost of prefill from weights."""
    torch.cuda.synchronize(); torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats()
    base = torch.cuda.memory_allocated()
    with torch.no_grad():
        model(**inputs, use_cache=True)
    torch.cuda.synchronize()
    return torch.cuda.memory_allocated() - base   # includes prefill activations; cross-check
     w/ formula

def gpu_energy_joules(fn):
    """Desktop NVIDIA energy via the NVML counter (Volta+). pip install nvidia-ml-py"""
    import pynvml
    pynvml.nvmlInit(); h = pynvml.nvmlDeviceGetHandleByIndex(0)
    mj0 = pynvml.nvmlDeviceGetTotalEnergyConsumption(h)  # millijoules
    fn(); torch.cuda.synchronize()
    return (pynvml.nvmlDeviceGetTotalEnergyConsumption(h) - mj0) / 1000.0
```

Tip

On a **Jetson Orin**, energy comes from real on-board sensors: `sudo tegrastats -interval 100` (watch the `VDD_CPU_GPU_CV` rail, in mW), or `pip install jetson-stats` then read `jtop().power['tot']`. On a desktop GPU use the NVML counter above (more reliable than `nvidia-smi -query-gpu=power.draw` for short kernels).

Part IV — The staged plan

6 Milestones (do them in order)

Stage 0 — Setup & baseline

Goal: run the unmodified model and record every baseline metric. **Produce:** a table of TTFT, TPOT, E2E, throughput, peak memory, measured KV bytes (vs the analytic formula), energy/token, and baseline WikiText-2 perplexity, for 1B at a few context lengths.

Stage 1 — Does the low-rank premise hold? (the cheap, essential first result)

Goal: extract K, V per layer from one prefill pass and plot the *singular-value spectrum* and *energy-retained-vs-rank*. No attention surgery needed. **Produce:** (i) spectrum curves per layer; (ii) energy-vs-rank for *keys vs values*; (iii) energy-vs-rank by *layer depth*. **Expected (from literature, but verify):** keys much more compressible than values [12]; depth- and RoPE-dependence are *open and contested* [12, 24] — a real chance to contribute a clean measurement.

Stage 2 — Accuracy impact (offline, no attention surgery)

Goal: prefill the same prompt with a full cache and with a rank-r cache; compare the next-token distributions (KL divergence and top-1 agreement) as a function of r. **Produce:** a KL-vs-r curve; the smallest r preserving the argmax token.

Stage 3 — Online integration & real cost

Goal: generate with a `LowRankCache` and measure TPOT, E2E, throughput, peak memory, energy. **Produce:** the honest latency/memory numbers — *including* whether naive reconstruction makes decoding *slower* (§13).

Stage 4 — Sweep & frontier

Goal: sweep r and context length; add a downstream task (GSM8K for reasoning, LongBench/RULER for long context). **Produce:** the compression-vs-accuracy-vs-latency Pareto frontier; separate keys-only, values-only, both.

Stage 5 — Baselines & tensor extension

Goal: compare against KIVI 2-bit quantization [4] and the full cache; then try Tucker/TT (`tensorly`) over the per-layer head×token×feature tensor (granularity 3). **Produce:** your method placed on the same axes as an established baseline; a first "genuinely tensor" result.

Part V — Implementation

7 Environment

```
# Python 3.10+, an NVIDIA GPU. Your versions: transformers 4.57.6, torch 2.8.0
pip install "transformers==4.57.6" torch --upgrade
pip install datasets matplotlib numpy nvidia-ml-py tensorly
# gated model: request access on huggingface.co/meta-llama, then `huggingface-cli login`
# (or use an ungated mirror such as unsloth/Llama-3.2-1B)
```

Common pitfall

The KV-cache internal layout is **version-dependent**. On your 4.57.x, read a layer with the portable `cache[i]` (returns a `(K,V)` tuple) and `len(cache)` for the layer count; write in place with `cache.layers[i].keys`. Older ($\leq$4.53) code used `cache.key_cache[i]`; the v5 line removes `to/from_legacy_cache`. Stick to `cache[i]` / `len(cache)` in teaching code.

8 Load the model

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
NAME = "meta-llama/Llama-3.2-1B-Instruct"     # or unsloth/Llama-3.2-1B-Instruct
tok = AutoTokenizer.from_pretrained(NAME)
model = AutoModelForCausalLM.from_pretrained(NAME, dtype=torch.bfloat16, device_map="cuda").
    eval()
inputs = tok("Explain why the sky is blue.", return_tensors="pt").to(model.device)
```

9 Stage 1 — extract K/V and plot the spectrum

```
import matplotlib.pyplot as plt

with torch.no_grad():
    out = model(**inputs, use_cache=True)      # one prefill pass
cache = out.past_key_values                     # a DynamicCache

def energy_vs_rank(mat):                        # mat: [S, d]
    s = torch.linalg.svdvals(mat.float())       # singular values, descending
    cum = torch.cumsum(s**2, 0) / (s**2).sum()  # fraction of energy kept by top-k
    return cum                                  # cum[r-1] = energy kept at rank r

for i in [0, len(cache)//2, len(cache)-1]:      # shallow / middle / deep layer
    K, V = cache[i]                             # each [B, num_kv_heads, S, head_dim]
    Kmat = K[0, 0]                              # layer i, head 0 -> [S, head_dim]
    Vmat = V[0, 0]
    plt.plot(energy_vs_rank(Kmat).cpu(), label=f"layer {i} K")
    plt.plot(energy_vs_rank(Vmat).cpu(), '--', label=f"layer {i} V")
plt.axhline(0.95, color="gray", ls=":"); plt.xlabel("rank r"); plt.ylabel("energy retained")
plt.legend(); plt.title("Low-rank structure of LLaMA-3.2 KV (keys solid, values dashed)")
plt.savefig("stage1_spectrum.png", dpi=150)
```

Note (RoPE): the cache stores keys *after* rotary position embedding. So you are measuring *post-RoPE* low-rankness; the literature disagrees on whether RoPE raises or lowers key rank [12], so state which you measured. Pre-RoPE analysis is a worthwhile extension.

10 Stages 2–3 — a low-rank cache

The clean, correct pattern: subclass `DynamicCache`, and *after the prefill step* replace the stored keys/values with their rank-r reconstruction; decode tokens append exactly (a small exact "tail").

```
from transformers import DynamicCache

def low_rank_reconstruct(x, r):
    """x: [B, H, S, D] -> rank-r reconstruction of each [S, D] slice (dense, same shape)."""
    B, H, S, D = x.shape
    r = min(r, S, D)
    U, s, Vh = torch.linalg.svd(x.float().reshape(B * H, S, D), full_matrices=False)
```

```
    xhat = (U[..., :r] * s[..., :r].unsqueeze(-2)) @ Vh[..., :r, :]
    return xhat.reshape(B, H, S, D).to(x.dtype)

class LowRankCache(DynamicCache):
    def __init__(self, rank=32, compress_keys=True, compress_values=True, **kw):
        super().__init__(**kw)
        self.rank = rank; self.ck = compress_keys; self.cv = compress_values
        self._done = set()
    def update(self, key_states, value_states, layer_idx, cache_kwargs=None):
        keys, values = super().update(key_states, value_states, layer_idx, cache_kwargs)
        if layer_idx not in self._done and keys.shape[-2] > 1:   # once, right after prefill
            if self.ck: keys   = low_rank_reconstruct(keys,   self.rank)
            if self.cv: values = low_rank_reconstruct(values, self.rank)
            self.layers[layer_idx].keys   = keys                 # write back (4.54+)
            self.layers[layer_idx].values = values
            self._done.add(layer_idx)
        return keys, values

# Stage 2: accuracy of the FIRST generated token, full vs rank-r
def next_logits(cache_ctor):
    c = cache_ctor()
    with torch.no_grad():
        o = model(**inputs, past_key_values=c, use_cache=True)
    return o.logits[0, -1].float()

full = next_logits(lambda: DynamicCache(config=model.config))
for r in [4, 8, 16, 32, 64]:
    lr = next_logits(lambda: LowRankCache(rank=r, config=model.config))
    kl = torch.nn.functional.kl_div(lr.log_softmax(-1), full.softmax(-1), reduction="sum")
    agree = (full.argmax() == lr.argmax()).item()
    print(f"r={r:3d}  KL={kl:.4f}  top1_agree={agree}")

# Stage 3: real generation cost with the compressed cache
def gen(cache_ctor, n=64):
    return model.generate(**inputs, do_sample=False, max_new_tokens=n,
                          past_key_values=cache_ctor())
print("dense  ", time_ms(lambda: gen(lambda: DynamicCache(config=model.config))))
print("rank-16", time_ms(lambda: gen(lambda: LowRankCache(rank=16, config=model.config))))
```

Tip

Memory vs latency, made explicit. `LowRankCache` above reconstructs *dense* tensors, so it measures *accuracy* and the *reconstruction time* but saves no memory. To actually save memory, store the factors A, B (and C, D for V) instead of $\hat{K}$, and reconstruct inside `update()` on read — memory drops to $r(S+d)$ per head, at the cost of a reconstruction each step. To save memory *and* avoid reconstruction, compute attention directly from the factors ($qK^\top \approx (qB)A^\top$) — that is the research frontier (compressed-domain attention), out of scope here but the natural next step.

11 Perplexity (baseline quality, Stage 0)

The canonical strided-window protocol [32]:

```
from datasets import load_dataset
def perplexity(model, tok, max_length=4096, stride=1024, device="cuda"):
    data = load_dataset("wikitext", "wikitext-2-raw-v1", split="test")
    enc = tok("\n\n".join(data["text"]), return_tensors="pt")
    seq_len = enc.input_ids.size(1); nlls = []; prev = 0
    for begin in range(0, seq_len, stride):
        end = min(begin + max_length, seq_len)
```

```
        trg = end - prev
        ids = enc.input_ids[:, begin:end].to(device)
        tgt = ids.clone(); tgt[:, :-trg] = -100          # only last `trg` tokens count
        with torch.no_grad():
            nlls.append(model(ids, labels=tgt).loss)
        prev = end
        if end == seq_len: break
    return torch.exp(torch.stack(nlls).mean()).item()
```

This scores the *baseline* model (teacher-forced, no growing cache). Measuring compression's effect on perplexity requires streaming autoregressive scoring with the compressed cache — a worthwhile Stage 4+ extension; for a first pass, judge compression by Stage 2 KL and Stage 4 task accuracy.

12 Stage 5 — per-layer Tucker (granularity 3, the tensor step-up)

Once the matrix case works, compress the per-layer 3-D tensor $[H \times S \times d]$ with Tucker, which shares a subspace *across the 8 heads* — a gain a per-head SVD cannot capture. Use `tensorly` (`pip install tensorly`), and keep it *per layer* (never stack the layer axis — see Part I).

```
import tensorly as tl
from tensorly.decomposition import tucker
tl.set_backend("pytorch")

K, V = cache[layer_idx]                 # K: [B, H, S, D]
Kten = K[0].float()                     # [H, S, D] = heads x tokens x head_dim

r_h, r_s, r_d = 8, 128, 16              # per-mode ranks (<= H, S, D); r_h < 8 shares across
    heads
core, factors = tucker(Kten, rank=[r_h, r_s, r_d])
Khat = tl.tucker_to_tensor((core, factors))          # reconstruction, shape [H, S, D]
rel_err = (Khat - Kten).norm() / Kten.norm()

dense      = Kten.numel()
compressed = core.numel() + sum(f.numel() for f in factors)
print(f"Tucker rel_err={rel_err:.4f}  compression={dense / compressed:.2f}x")
```

Compare this compression-vs-error point against granularity (1) (per-head SVD) at *matched* memory: if Tucker wins, the cross-head sharing is real structure worth exploiting. (`tensor_train` gives the TT variant; reconstruct with `tl.tt_to_tensor`.)

Part VI — Interpreting your results

13 What to expect (and the honest lessons)

1. **Keys compress, values resist.** Expect key spectra to decay fast and value spectra to be flatter; ShadowKV reports values are *not* low-rank [12], which is why it keeps low-rank keys but offloads values. If you see this, it is a real, reportable finding — and it argues for compressing keys and values *differently*.

2. **Depth and RoPE are open questions.** The literature disagrees on whether early or deep layers are more compressible, and on whether RoPE raises or lowers key rank [12, 24]. Your clean single-model measurement is a genuine contribution — just state your setup (which

model, context length, pre- vs post-RoPE).

3. **Naive low-rank can be *slower*.** This is the most important lesson. If you store factors and reconstruct dense $\hat{K}$ every step, you pay an SVD/matmul per step; on a small head dim ($d = 64/128$) the dense attention was already cheap, so reconstruction overhead can exceed the savings, and TPOT *goes up*. This is not a failure of your experiment — it is precisely why the field moves to (a) storing factors for *memory* while accepting some latency, (b) computing attention *in the compressed domain* (no reconstruction), or (c) query-aware sparse retrieval. Documenting the crossover (r, context length) at which low-rank helps or hurts is a publishable result on its own.

4. **Reconstruction error underestimates task damage.** A rank that looks fine on $\|K-\hat{K}\|$ and even on perplexity can still hurt GSM8K, because chained reasoning compounds per-step error [5]. Always keep a reasoning task in the loop.

Key idea

These "negative" findings are the bridge to the research program: the memory-wall poster's pipeline [1] plus *compressed-domain attention* (compute from factors, never reconstruct) is exactly the fix for lesson 3. Your single-device evaluation is the evidence base that motivates it.

Part VII — Related work

14 The landscape (grouped, with code where public)

KV quantization: KIVI [4] (2-bit; keys per-channel, values per-token; code); GEAR [5] (quant + low-rank error + sparse outliers; code); DecoQuant [6] (tensor-decomposition-assisted low-bit KV; code).

Token eviction / attention sparsity: H2O [7] (heavy hitters); SnapKV [8]; Quest [9] (query-aware page selection); RocketKV [10] (two-stage eviction + sparse attention).

Low-rank / tensor KV (closest to this project): Palu [11] (low-rank projection of KV; code); ShadowKV [12] (low-rank keys on GPU, values offloaded; code); Eigen Attention [13]; MLA / DeepSeek-V2 [14] (trained-in low-rank latent KV); JoLT [15] (partial Tucker of the KV tensor + JL residual — the explicit tensor version; recent preprint).

Attention-output-error theory (why not $\|K - \hat{K}\|$): KDEformer [16]; Ada-KV [17] (error bounded by evicted attention mass); the total-variation / missed-mass identity [18] (recent preprint).

Tensor foundations: Robust PCA (low-rank + sparse) [19]; CP/Tucker [20]; Tensor-Train [21]; low-rank + sparse weights (LoSparse) [22]; low-rank attention (Linformer) [23].

Systems & the base pipeline: FlashAttention [25] (the online-softmax primitive); vLLM / PagedAttention [26] (the standard KV memory manager you are implicitly compared against); the FSU pipelined-inference poster [1], HexGen [2], Jupiter [3].

Evaluation: WikiText-2 [27]; C4 [28]; LongBench [29]; RULER [30]; GSM8K [31]; the HF perplexity protocol [32].

Tip

Recent (2025–2026) preprints marked as such (JoLT, the TV/missed-mass identity) are verified to exist on arXiv but are not yet peer-reviewed — read the PDF before leaning on their exact numbers. Everything else here is published.

Part VIII — Deliverables & checklist

15 What to produce

- **Plots:** (1) singular-value / energy-vs-rank spectra, keys vs values, by layer; (2) KL-divergence vs rank; (3) compression-vs-accuracy Pareto (per keys/values/both); (4) TPOT and peak-memory vs rank and context length; (5) energy/token vs rank.
- **Tables:** baseline metrics; the crossover r where low-rank helps/hurts latency; your method vs KIVI 2-bit vs full cache.
- **One figure that tells the story:** accuracy retained (y) vs compression ratio (x), with a second panel of latency vs compression — this is the paper's money plot.

16 Reproducibility checklist

☐ Fixed seeds; greedy decoding for all accuracy comparisons.

☐ `cuda.synchronize` / CUDA events around every timing; warmup done.

☐ Median and P95 reported over $\geq$50 iters.

☐ KV bytes reported both analytically and measured.

☐ Same tokenizer and context length across compared runs (perplexity is tokenizer-dependent).

☐ Pre- vs post-RoPE stated for any spectrum plot.

☐ A reasoning task (GSM8K) included, not only perplexity.

☐ transformers / torch versions recorded in the writeup.

References

[1] H. Zhou, X. Zhang. *Poster: Distributed Pipelined Collaborative Inference for LLMs on Edge Device Clusters.* ACM HotMobile 2026.

[2] Y. Jiang et al. *HexGen: Generative Inference of LLM over Heterogeneous Environment.* ICML 2024. arXiv:2311.11514.

[3] S. Ye et al. *Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices.* INFOCOM 2025. arXiv:2504.08242.

[4] Z. Liu et al. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024. arXiv:2402.02750.

[5] H. Kang, Q. Zhang, G. Jeong, T. Krishna, T. Zhao. *GEAR: An Efficient Error Reduction Framework for KV Cache Compression.* PMLR v262, 2024. arXiv:2403.05527.

[6] P. Liu et al. *Unlocking Data-free Low-bit Quantization with Matrix Decomposition for KV Cache Compression (DecoQuant).* ACL 2024. arXiv:2405.12591.

[7] Z. Zhang et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs.* NeurIPS 2023. arXiv:2306.14048.

[8] Y. Li et al. *SnapKV: LLM Knows What You Are Looking For Before Generation.* arXiv:2404.14469, 2024.

[9] J. Tang et al. *Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference.* ICML 2024. arXiv:2406.10774.

[10] P. Behnam et al. *RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression.* ICML 2025. arXiv:2502.14051.

[11] J. Chang et al. *Palu: Compressing KV-Cache with Low-Rank Projection.* ICLR 2025. arXiv:2407.21118.

[12] H. Sun et al. *ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference.* ICML 2025. arXiv:2410.21465.

[13] U. Saxena et al. *Eigen Attention: Attention in Low-Rank Space for KV Cache Compression.* EMNLP 2024 Findings. arXiv:2408.05646.

[14] DeepSeek-AI. *DeepSeek-V2* (Multi-head Latent Attention: trained-in low-rank KV). arXiv:2405.04434, 2024.

[15] *A JoLT for the KV Cache: Near-Lossless KV Cache Compression via Joint Tucker and JL-Residual Allocation.* arXiv:2607.12550, 2026 (preprint).

[16] A. Zandieh, I. Han, M. Daliri, A. Karbasi. *KDEformer: Accelerating Transformers via Kernel Density Estimation.* ICML 2023. arXiv:2302.02451.

[17] Y. Feng et al. *Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation.* arXiv:2407.11550, 2024.

[18] A. Tzachristas et al. *A Mathematical Theory of Top-k Sparse Attention via Total Variation Distance.* arXiv:2512.07647, 2025 (preprint).

[19] E. J. Candès, X. Li, Y. Ma, J. Wright. *Robust Principal Component Analysis?* J. ACM 58(3):11, 2011.

[20] T. G. Kolda, B. W. Bader. *Tensor Decompositions and Applications.* SIAM Review 51(3):455–500, 2009.

[21] I. V. Oseledets. *Tensor-Train Decomposition.* SIAM J. Sci. Comput. 33(5):2295–2317, 2011.

[22] Y. Li et al. *LoSparse: Structured Compression of LLMs based on Low-Rank and Sparse Approximation.* ICML 2023. arXiv:2306.11222.

[23] S. Wang et al. *Linformer: Self-Attention with Linear Complexity.* arXiv:2006.04768, 2020.

[24] R. Zhang et al. *LoRC: Low-Rank Compression for LLMs KV Cache with a Progressive Compression Strategy.* arXiv:2410.03111, 2024.

[25] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135.

[26] W. Kwon et al. *Efficient Memory Management for LLM Serving with PagedAttention (vLLM).* SOSP 2023. arXiv:2309.06180.

[27] S. Merity et al. *Pointer Sentinel Mixture Models* (WikiText). arXiv:1609.07843, 2016.

[28] C. Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* (C4). arXiv:1910.10683, 2019.

[29] Y. Bai et al. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.* ACL 2024. arXiv:2308.14508.

[30] C.-P. Hsieh et al. *RULER: What's the Real Context Size of Your Long-Context Language Models?* COLM 2024. arXiv:2404.06654.

[31] K. Cobbe et al. *Training Verifiers to Solve Math Word Problems (GSM8K).* arXiv:2110.14168, 2021.

[32] Hugging Face. *Perplexity of fixed-length models.* https://huggingface.co/docs/transformers/en/perplexity.